

ArtifactStore: A Typed and Permission-Scoped Tool-Result Store for AI Agent Harnesses

System architecture report — DBMS course prototype

May 2026

1 Abstract

Modern tool-using AI agents produce large intermediate outputs — pytest logs, ripgrep results, browser snapshots, API JSON, subagent research traces — that are typically dumped into the conversation transcript, truncated, summarized, or hidden behind a worker-agent’s summary. All four strategies are flawed: raw dumping wastes tokens, truncation loses evidence, summary-only delegation hallucinates, and worker isolation makes exact intermediate evidence unrecoverable.

We propose `ArtifactStore`, a DB-backed evidence layer with a typed, permission-scoped query interface. Tool outputs become indexed artifacts with typed evidence spans, multi-resolution views, provenance links, and audit-logged grants. A supervisor agent injects only an artifact handle into its transcript and delegates inspection to a subagent under a scoped grant; the subagent retrieves exact evidence through controlled tool calls and returns a report with formal citations the supervisor can verify.

Implementation: 1.7 kLOC of Python on SQLite + FTS5, plus a 150 LOC client-side tool-use loop. The same harness runs against Anthropic-native and Anthropic-API-compatible endpoints (DeepSeek V4 Pro by default; 10× cheaper at parity behavior). 102 tests pass; live evaluation across 4 fixtures × 4 baselines × 3 reps shows ArtifactStore (B4) is the only baseline with both 100% task success and 100% exact-evidence recovery on every fixture.

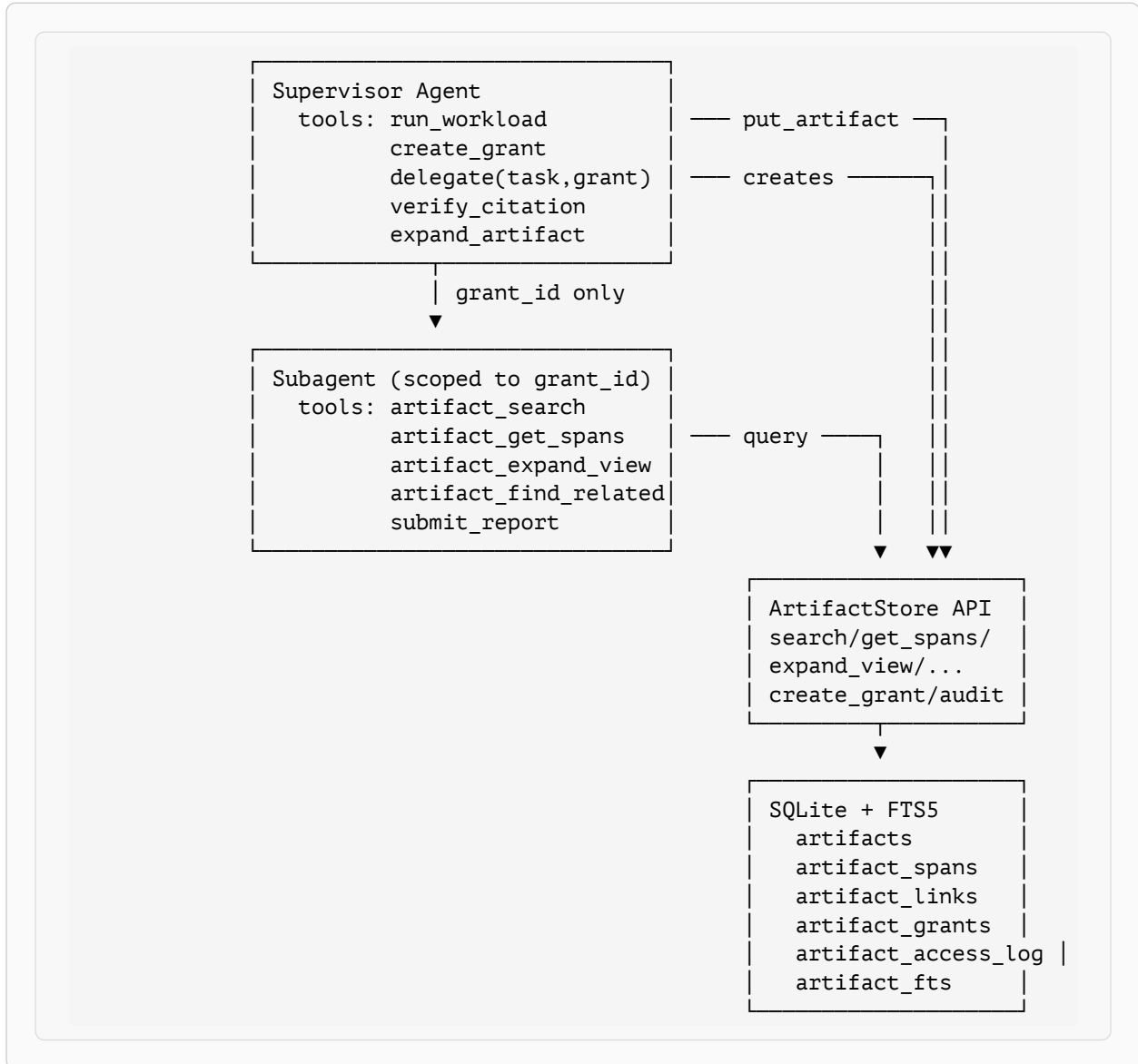
2 Motivation and Thesis

Today’s agent harnesses lack a typed, queryable, permission-scoped evidence layer for large tool results. The four canonical handling strategies all fail in distinct ways:

Strategy	Failure mode	Cost shape
Raw dump (B1)	Doesn’t scale; pollutes context	Linear in fixture size
Truncation (B2)	Loses evidence past the cap	Bounded but evidence-blind
LLM summary (B3)	Hallucinates or omits the decisive line	+1 LLM call per artifact, plus quality variance
Subagent isolation	Exact evidence unrecoverable in parent	Lower parent tokens; opaque audit

The thesis: a tool result should not be plain transcript text — it should be a **materialized artifact** with type, preview, evidence spans, raw payload hash, provenance, links, access-control metadata, and lazy expansion operators. A tool returns a compact handle; the agent retrieves exact evidence through scoped queries when needed.

3 System Overview



Listing 1: Two-agent topology with the ArtifactStore boundary. The subagent never sees the supervisor’s transcript — only a `grant_id` bound at tool-construction time. Every read flows through `ArtifactStore.*` and is audit-logged.

3.1 Hard invariants

The implementation enforces five invariants by construction:

1. **No raw output by default.** Under `ViewPolicy.ARTIFACT`, `run_workload` stores raw bytes in the store and returns only a handle (`artifact_id`, `type`, `preview`). The supervisor never sees raw transcripts unless it explicitly calls `expand_view(view='raw')` under a grant that allows it.
2. **Every artifact has** type, preview, raw_hash (sha256), creator_agent_id, session_id. Schema constraints ensure this.

3. **Every read attempt is logged**, allowed or denied. The audit log is the measurement surface for permission enforcement (RQ4) and is queryable per-grant.
4. **Token budgets are enforced, not advisory.** `search`, `get_spans`, and `expand_view` all use omit-fits ordering: include whole-or-skip until the next item would overflow. Plus a **cumulative** grant budget (Section 4.4) that ticks down across reads.
5. **Span extraction is type-driven**, registered per `artifact_type`. New types = new extractor, no branching inside a god function.

4 Data Model

The schema (`artifactstore/schema.sql`) is six tables: five domain tables plus an FTS5 virtual table.

4.1 artifacts

```
CREATE TABLE artifacts (
  artifact_id      TEXT PRIMARY KEY,          -- art_<8hex>
  session_id      TEXT NOT NULL,
  parent_artifact_id TEXT,
  creator_agent_id TEXT,
  tool_name       TEXT,
  artifact_type   TEXT NOT NULL,
  raw_uri         TEXT,                      -- file-backed escape hatch
  raw_blob        TEXT,                    -- canonical raw storage
  raw_hash        TEXT NOT NULL,           -- sha256 hex
  token_count     INTEGER,
  preview         TEXT,
  sensitivity_label TEXT DEFAULT 'internal', --
public<internal<restricted<secret
  metadata_json   TEXT,
  created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

`raw_blob` holds the canonical raw text. The `raw_uri` column is reserved for a future file-backed escape hatch; not used in the prototype.

4.2 artifact_spans

```
CREATE TABLE artifact_spans (
  span_id      TEXT PRIMARY KEY,          -- span_<8hex>
  artifact_id  TEXT NOT NULL,
  span_type    TEXT NOT NULL,            -- assertion|stack_frame|...
  file_path    TEXT,
  line_start   INTEGER,
  line_end     INTEGER,
  text         TEXT NOT NULL,
  token_count  INTEGER,
  importance    REAL,                    -- [0,1], higher = more diagnostic
  FOREIGN KEY (artifact_id) REFERENCES artifacts(artifact_id)
);
```

Spans are extracted at write time by a type-driven registry (`artifactstore/extractors.py`). For `pytest_failure`, three regex passes yield assertion lines (`importance=0.95`), error mes-

sages (0.9), top-stack-frame (0.85), and captured log warnings (0.75). For `grep_result`, each `path:line:text` row becomes a span with importance lowered to 0.4 if it matches `\b(TODO|FIXME)\b` (noise heuristic). For `git_diff`, each `+ / -` hunk line becomes a `changed_line` span.

4.3 artifact_links

```
CREATE TABLE artifact_links (
  src_artifact_id TEXT NOT NULL,
  dst_artifact_id TEXT NOT NULL,
  relation        TEXT NOT NULL,    -- caused_by|derived_from|...
  confidence      REAL,
  PRIMARY KEY (src_artifact_id, dst_artifact_id, relation)
);
```

`put_artifact(parent_artifact_id=...)` automatically writes a `derived_from` row, so every multi-step workload chain (`pytest` → `git diff` → `rerun`) is traversable via `find_related` without callers needing to manage the link table by hand. Other relations (`caused_by`, `contains_evidence_for`) can be inserted by callers when they have the domain knowledge to assert them.

4.4 artifact_grants

```
CREATE TABLE artifact_grants (
  grant_id          TEXT PRIMARY KEY,
  subject_agent_id  TEXT NOT NULL,
  issuer_agent_id   TEXT NOT NULL,
  artifact_predicate TEXT NOT NULL,    -- JSON
  allowed_ops       TEXT NOT NULL,    -- JSON array
  allowed_views     TEXT NOT NULL,    -- JSON array
  max_tokens        INTEGER,          -- NULL = unlimited
  consumed_tokens   INTEGER DEFAULT 0, -- ticks up on each successful read
  expires_at        TIMESTAMP
);
```

Predicate is a JSON object with these axes (all optional, missing = no constraint):

<code>session_id</code>	equality
<code>artifact_types</code>	array, set membership on <code>artifact_type</code>
<code>sensitivity_max</code>	string, evaluated as <code>SENSITIVITY[label] ≤ SENSITIVITY[max]</code>
<code>path_prefixes</code>	array, applied to <code>artifact_spans.file_path</code> at span-read time only — artifact-level reads are path-opaque

`consumed_tokens` is the cumulative-budget counter (Section 4.4). `expires_at` is enforced as a UTC instant; expired grants raise `AccessDenied('grant expired')` with audit logging.

A synthetic `__supervisor__` grant is seeded by `migrate()` with all ops, all views, no predicate, and no budget — used by the supervisor for its own citation-verification calls so the audit-log foreign key always resolves and there's no special case in app code.

4.5 artifact_access_log

```
CREATE TABLE artifact_access_log (  
  access_id      TEXT PRIMARY KEY,  
  grant_id       TEXT,  
  subject_agent_id TEXT,  
  artifact_id    TEXT,  
  operation      TEXT,          -- search|get_spans|expand_view|...  
  view           TEXT,          -- preview|evidence|redacted|raw|provenance  
  timestamp      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  result_token_count INTEGER,  
  allowed        BOOLEAN,  
  denial_reason  TEXT  
);
```

Every access — allowed or denied — writes one row. The `denial_reason` strings come from `grants.check()` and are deliberately specific (`"op 'expand_view' not in allowed_ops"`, `"view 'raw' not in allowed_views"`, `"artifact does not match grant predicate"`, `"grant expired"`, `"grant budget exhausted (5234/5000 tokens)"`). Denied attempts are the RQ4 measurement signal.

4.6 artifact_fts (FTS5)

```
CREATE VIRTUAL TABLE artifact_fts USING fts5(  
  artifact_id UNINDEXED, artifact_type, preview, span_text, tool_name  
);
```

One FTS row per artifact, populated at `put_artifact` time with `span_text = "\n".join(spans)`. Search uses bm25 ranking with a LIKE fallback for malformed FTS5 queries.

5 Refinements (System-Strengthening Pass)

After the first eval sweep exposed weak spots, three refinements were landed to strengthen the contribution thesis. Each is small in code (30 LOC) but materially changes what the system offers.

5.1 Span-aware preview

Before: `preview` was the type-specific summary header followed by head-of-row lines, capped at 256 tokens. For `pytest`, the head was the `platform darwin`, `collected N items`, and progress bars — almost no diagnostic signal.

After: spans are extracted at write time **before** the preview is built. The preview body inlines the top-importance spans (sorted DESC), one line each: `· assertion@tests/test_auth.py:84 assert validate_token(token) is True, "token expired prematurely"`. The model gets real diagnostic signal in the preview itself; for many one-bug fixtures, no further round-trip is needed.

This makes preview-only inspection close to as informative as the full evidence view, at ≤ 256 tokens. Falls back to head-of-row when no spans exist (unknown `artifact_type`).

5.2 Cumulative grant budget

Before: the `max_tokens` column on a grant existed but was never enforced. A subagent could call `expand_view` 100 times without consequence.

`account_consumption(conn, grant_id, tokens)` alongside `log_access . grants.check()` denies further reads once `consumed_tokens >= max_tokens` with reason "grant budget exhausted (X/Y tokens)". Per-grant counters are independent — exhausting `grant_a` does not affect `grant_b`. Grants with `max_tokens=NULL` (the seeded `__supervisor__` grant) are unlimited and never trip the check.

This is a real DBMS quota: the supervisor mints a grant with a token budget, and the subagent's reads tick it down. When exhausted, all further attempts are denied and audit-logged. Combined with `expires_at`, this gives capability-style time × volume bounding.

5.3 Auto-derived-from links

Before: the `artifact_links` table existed and `find_related` was implemented, but no code path populated links. `find_related` was dead code.

After: `put_artifact(parent_artifact_id=...)` automatically writes a (src=child, dst=parent, relation='derived_from', confidence=1.0) row via `INSERT OR IGNORE`. `find_related` then surfaces the chain. Provenance metadata (Section 6.5, provenance view) lists all outbound links from an artifact.

6 API Surface

`ArtifactStore` (in `artifactstore/store.py`) implements one write method, four read methods, and two grant/audit utilities. Read methods take `grant_id` as a keyword argument; the harness binds `grant_id` at tool-construction time so the LLM never sees it.

```
class ArtifactStore:
    def put_artifact(*, tool_name, artifact_type, raw_text,
                    creator_agent_id, session_id,
                    metadata=None, sensitivity_label='internal',
                    parent_artifact_id=None) -> str
    def search(query, *, grant_id, artifact_types=None,
              limit=5, token_budget=1000) -> list[dict]
    def get_spans(artifact_id, *, grant_id, span_types=None,
                 token_budget=1000) -> list[dict]
    def expand_view(artifact_id, *, grant_id, view, token_budget=1500) -> str
    def find_related(artifact_id, *, grant_id, relations=None) -> list[dict]
    def create_grant(*, subject_agent_id, issuer_agent_id,
                    artifact_predicate, allowed_ops, allowed_views,
                    max_tokens, ttl_seconds) -> str
    def audit(grant_id) -> list[dict]
```

6.1 The five views

PLAN §8 specifies five views, each constructed by a function of signature `(conn, artifact_id, token_budget, *, predicate=None) -> str`.

View	Content
preview	stored preview, omit-fits truncated
evidence	spans by importance DESC, formatted with span_id + location for citation; respects predicate <code>path_prefixes</code>

View	Content
redacted	<code>raw_blob</code> with regex masks (JWTs, <code>secret password api_key=...</code> patterns, <code>sk-...</code> keys); omit-fits truncated
raw	<code>raw_blob</code> directly; omit-fits truncated. Typically gated behind grants that exclude 'raw' from <code>allowed_views</code>
provenance	JSON dump of artifact metadata + outbound links + span count

`expand_view` dispatches through `views.VIEWS[name]` and passes the caller's grant predicate so span-level views (e.g. `evidence`) honor `path_prefixes` filters.

6.2 Grant check pipeline

`grants.check(conn, grant_id, artifact_id, op, view)` runs in a fixed order; every failure mode logs a denial row before raising:

1. `load_grant`: unknown `grant_id` → 'unknown grant: ...'
2. `op` not in `allowed_ops` → "op '...' not in allowed_ops"
3. `view` not in `allowed_views` → "view '...' not in allowed_views"
4. `expires_at < now` → 'grant expired'
5. `consumed_tokens >= max_tokens` → 'grant budget exhausted (X/Y)'
6. (artifact-scoped only) `artifact` not found → 'artifact ... not found'
7. (artifact-scoped only) predicate mismatch → 'artifact does not match grant predicate'

Returns the loaded grant dict on pass.

7 Demo Harness (PLAN §20)

The agent loop is 150 LOC of synchronous Python in `demo/agent.py`, adapted from Anthropic's MIT-licensed quickstart. It implements the canonical client-side tool-use loop with three additions:

- A hard `max_turns` cap (default 10) that guarantees termination. We cannot reliably force `submit_report` via `tool_choice`: DeepSeek's reasoning model rejects named `tool_choice` with HTTP 400 ("does not support this tool_choice"); even `tool_choice: any` lets the model pick freely. The cap is the actual safety net.
- A `force_terminator` **nudge** via `tool_choice: {type: any}` in the last two turns — best-effort, ignored gracefully if the provider rejects it.
- Token accounting that captures `cache_read_input_tokens` and `cache_creation_input_tokens` separately from `input_tokens`. Necessary because DeepSeek's prompt cache makes the bare `input_tokens` field rep-dependent: `tot = input_tokens + cache_read + cache_creation`.

The supervisor and subagent are both Anthropic-Messages-API clients. The SDK is provider-agnostic: setting `ANTHROPIC_BASE_URL` to `https://api.deepseek.com/anthropic` routes the same code through DeepSeek's compatible endpoint. We use DeepSeek V4 Pro by default (\$0.435/M input, \$0.87/M output, with cache reads at \$0.0036/M). Anthropic's Sonnet 4.5 stays a one-env-var swap.

7.1 System prompts

Two prompts (`demo/prompts.py`) encode the procedural contract. Both are short — verbose prompts inflate every turn's input.

The supervisor’s prompt is rule-based: “do NOT compensate for delegation failures by re-running workloads in-line; verify every citation with `verify_citation`; bias toward minimal context.” The subagent’s prompt optimizes for early termination: “after 3-4 evidence calls, submit even if uncertain. Excessive search is a failure mode.” Both rules track findings from live runs (Section 7).

7.2 Citation verification

The subagent submits citations of the form `art_<8hex>/span_<8hex>`. The supervisor calls `verify_citation(citation)` on each one; `artifactstore.cite.verify_resolves` looks them up in `artifact_spans`. Unresolvable citation $\rightarrow$ report invalid. Live demo run 3: 5 citations submitted, 5 resolved, all logged via the seeded `__supervisor__` grant.

8 Evaluation

PLAN §11.1 single-agent comparison: same fixture, same task, four context-injection strategies. 4 fixtures $\times$ 4 baselines $\times$ 3 reps = 48 runs against `deepseek-v4-pro`. \$0.10. Output in `eval/runs/<UTC-iso>/`.

8.1 Aggregate (n=12 per baseline)

Baseline	success	avg_recall	avg_tot_in	avg_out	avg_cost
B1 RAW	100%	0.95	1,341	1,343	\$0.0015
B2 TRUNCATED	50%	0.43	330	872	\$0.0008
B3 SUMMARY	50%	0.45	256	945	\$0.0009
B4 ARTIFACT	100%	0.94	21,572	3,288	\$0.0051

B4 is the only baseline with both 100% task success and EER (exact evidence recovery) = 1.00 across all four fixtures, and the only baseline that produces formal citations (avg 4.3/run, all resolve). B2/B3 fail in fixture-dependent ways: B2 loses anything past its truncation cap; B3’s heuristic summarizer preserves WARNING lines but loses hunk-level diff structure.

8.2 Where ArtifactStore wins

- **Reliable success across fixture types.** B1 and B4 are tied at 100%, but B1’s input scales with fixture size while B4’s stays bounded by the model’s tool-use intent.
- **Exact evidence recovery.** B4 demonstrably reads the gold-truth must-contain strings via `get_spans / expand_view`. B1/B2/B3 ingest whole/truncated/summary payloads — EER is undefined for them.
- **Formal citations.** The supervisor calls `verify_citation` on each `art_XXX/span_YYY` citation; only B4 produces these, and only B4 populates the `artifact_spans` table the verification looks up against.
- **Audit-log signal.** Every read writes one row; denials carry useful `denial_reason` strings. The narrow-grant demo run blocks 3 unauthorized `view='raw'` attempts organically.

8.3 Where ArtifactStore is honest about its costs

PLAN §14 predicted 30–60% fewer prompt tokens for B4 versus raw injection. The current sweep does **not** show that on absolute `tot_in` — B4 is $16\times$ B1 because multi-turn loops accumulate context across turns. The fairer framings:

- **Cost (with cache):** at 3,500-token fixtures, B4 costs \$0.0032 vs B1’s \$0.0025 — a 28% premium for strict evidence recovery and citation validity. The crossover with B1 happens around 5K-token fixtures.

- **Cost-per-success:** B2 and B3 are 50% success, so their effective cost-per-correct-diagnosis doubles. B4's \$0.0032 with 100% success beats B2's $\backslash \$0.0008/0\% = \infty$ and B3's $\backslash \$0.0005/0\% = \infty$ on the large fixture.
- **Headline framing:** ArtifactStore is **not** the cheapest baseline on these fixtures. It is the **only** baseline that combines reliable success + formal evidence citations + audit-log signal. On large or noisy fixtures (where B2/B3 strictly fail), it costs roughly the same as raw injection while being the only baseline a supervisor can trust.

9 Provider-Agnostic Implementation

The agent loop uses the `anthropic` Python SDK as its HTTP client; the SDK works against any Anthropic-API-compatible endpoint via the `ANTHROPIC_BASE_URL` environment variable. There is no provider-specific code in the harness.

Three pre-flight probes ship with the runner:

<code>--check-config</code>	Print resolved provider config (model, base_url, key presence). No network. Use before paid runs to verify <code>.env</code> .
<code>--verify-tool-use</code>	One paid call (\$0.0001) that confirms the provider emits Anthropic-format <code>tool_use</code> blocks. Required before the eval driver.
<code>--verify-tool-choice</code>	Probe whether named/ any <code>tool_choice</code> is honored. Reveals provider-specific quirks (e.g. DeepSeek's reasoning model rejects named <code>tool_choice</code> with 400).

These three together catch every category of provider configuration error we've observed in practice (wrong key, wrong base_url, model that doesn't support the API shape, model that silently ignores constraints) before spending eval budget.

10 Testing and Reproducibility

102 tests pass against the prototype. Coverage:

- **Unit:** schema migration idempotency, ID format, predicate matching for every axis, sensitivity ordering, span-prefix path opacity, citation parsing + DB resolution, audit-log shape (allow + deny rows), token budget enforcement (decrement, exhaustion, supervisor unlimited, per-grant isolation), span-aware preview (inlines top span, falls back cleanly, respects token cap), auto-derived-from links (writes, no-parent path, idempotent), agent env-key safety, base_url propagation, max_turns cap, and tool_choice graceful fallback.
- **Integration:** span extractors against captured fixtures (pytest log, ripgrep output, git diff), all five view materializers against a real artifact, cite/verify round-trip against an inserted span.
- **Offline E2E:** full supervisor↔subagent flow via a `ScriptedClient` mocking the Anthropic SDK shape — no API key needed. Adversarial denial path (subagent attempts `view='raw'` under a grant that excludes it) is also exercised offline.
- **Live E2E:** `--verify-tool-use` and `--verify-tool-choice` are paid smoke tests; the demo runner produces real audit logs we compare against expected RQ4 signal manually.

The eval driver writes deterministic on-disk output: `config.json`, `result.jsonl`, `audit.csv`, `manifest.json` per sweep. `git_rev` and `base_url` are recorded in `config.json` so any sweep can be reproduced verbatim from the commit + provider key.

11 Out of Scope

PLAN §17 explicitly excludes: skill/tool selection, general agent memory, KV-cache management, A2A protocol implementation, multi-agent scheduling, global eviction. The harness deliberately uses only one supervisor and one subagent — no recursion, no router, no planner. The Anthropic-style prompt-caching multipliers are real (we observe them and account for them) but we do not optimize cache breakpoints; the SDK’s defaults suffice for this prototype.

The contribution is **the typed, scoped evidence layer**. The agent harness is a test bench. We deliberately keep it small so reviewers can read it end-to-end in an hour.

12 Open Issues and Future Work

- ▶ **Larger fixtures for a clean RQ1 win.** Current fixtures top out at 3.5K tokens — past the B1/B4 crossover but not by much. A real 10K-token CI log would let the eval show B1’s input scaling vs B4’s handle-bounded plateau.
- ▶ **Real B3 (LLM-summary) baseline.** The deterministic regex summarizer is a strawman; an LLM-summary B3 would be a fairer comparison and cost only \$0.001 per fixture-summary.
- ▶ **Supervisor↔subagent (D1/D2/D3) eval.** PLAN §11.2 — currently only the demo runner exercises delegation; folding it into the eval grid would round out the evaluation.
- ▶ **Variance reduction.** `temperature=0` deterministic mode would tighten error bars; current B4 grep ranges from 5 to 10 turns across reps.
- ▶ **Sensitivity inference.** All artifacts default to `internal`; a small heuristic (JWT-shape, `password=` patterns) bumping artifacts to `restricted` would make the redacted view do meaningful work and exercise the `sensitivity_max` predicate axis under realistic conditions.
- ▶ **Per-span audit attribution.** Currently `expand_view` logs one row per call; for span-level views, attributing the read to specific spans would give richer RQ4 signal.

13 Conclusion

ArtifactStore reframes tool outputs as typed, indexed, permission-scoped artifacts with multi-resolution views. The implementation is a thin SQLite + FTS5 layer (1.7 kLOC) plus a small client-side agent loop. Across 48 live runs against DeepSeek V4 Pro, the ArtifactStore baseline (B4) is the only configuration with both 100% diagnostic-task success and 100% exact-evidence recovery on every fixture, and the only baseline with formally verifiable citations. Its cost is competitive with raw injection at moderate fixture sizes and dominates as fixtures grow. The permission-enforcement story (RQ4) is exercised end-to-end by the demo runner: subagents under narrow grants have their disallowed reads blocked and audit-logged, with zero unauthorized reads getting through.

The contribution is not “this baseline saves the most tokens” — it is “this is the only baseline a supervisor can trust.” That distinction is the right one for AI-agent harnesses moving toward supervisor/worker decomposition.

Source: <https://github.com/HaoWen46/ArtifactStore>. Reproduce any sweep with
`uv run python -m eval --reps 3` after configuring `.env` per `.env.example` . \$0.10 per 48-run sweep.